Evaluating Scalability and Latency in Distributed Hash Tables

Daryl Ho (1620058, dho@scu.edu)
Josh Kindarara (1620620, jkindarara@scu.edu)

ABSTRACT

Distributed Hash Tables (DHTs) form the backbone of scalable decentralized systems, including peer-to-peer (P2P) file sharing, distributed storage layers, and large-scale data retrieval infrastructures. This paper presents the design, implementation, and evaluation of Chord and Kademlia, the two widely studied and utilized DHT architectures;. We develop complete working implementations of both systems, including single-process and multi-process Chord as well as a multi-process Kademlia implementation. We evaluate their performance in terms of lookup efficiency, load balancing, fault-tolerance, routing complexity, and stabilization overhead. Our results confirm that Chord offers predictable logarithmic lookup latency as scale increases, while Kademlia provides lower latency and parallelized queries due to XOR-metric distance and k-bucket-based routing. We conclude with lessons learned, design tradeoffs, and implications for building scalable and fault-tolerant distributed systems.

I. PROBLEM

As distributed systems scale to accommodate increasing numbers of users, data, and resources, one of the challenges that becomes quickly apparent is scalability. Traditional client-server architectures struggle to efficiently manage growing workloads, since a centralized server becomes a bottleneck once the load becomes too large. When the number of nodes and requests grow, maintaining consistent performance and reliability through a server becomes increasingly difficult. This limitation has motivated many to come up with a decentralized approach to distributed systems, where no single node holds complete responsibility.

In large-scale decentralized systems, efficiently locating and retrieving data without relying on a centralized server remains a fundamental challenge. Many client-server architectures struggle with scalability, single points of failure, and load management [1]. Distributed Hast Tables (DHTs) address this problem by providing a fully decentralized mechanism for key-based lookup across a dynamic network of nodes.

In a DHT, data is distributed among participating nodes using consistent hashing so that each node is responsible for a portion of the key space. When a node joins or leaves, the system must rebalance keys such that the routing remains correct. The system must also perform efficient lookup algorithms in order to find items in large and rapidly changing distributed systems.

II. GOALS

The primary goal of this project is to design, implement, and rigorously evaluate two foundational structured peer-to-peer Distributed Hash Table (DHT) protocols: Chord and Kademlia. These systems serve as the backbone of decentralized key-value storage, enabling efficient lookup, insertion, and management of data without reliance on any centralized authority. As both protocols were developed to address core distributed system challenges, they provide an ideal basis for comparative study. We aim to deeply explore the algorithmic mechanisms that enable these protocols to maintain consistent, structured overlays in the presence of constantly changing network environments. We plan to accomplish this by implementing each protocol from scratch, including both single-process and multi-process versions for Chord and a full multi-process version for Kademlia.

A second core goal of this project is to evaluate how Chord and Kademlia perform under varying environmental conditions, specifically when the number of participating nodes and the amount of churn differ significantly. To accomplish this, the project includes experimental scenarios involving small, medium, and large-scale networks. Additionally, we assess system behavior under varying levels of churn, where nodes join or leave the network at differing frequencies. These experiments allow us to measure lookup latency, routing complexity, stabilization costs, and consistency behavior under realistic peer-to-peer dynamics. The project aims to reveal how each protocol adapts to growth, contraction, and instability by analyzing system performance in a wide range of controlled but diverse conditions.

III. MOTIVATION

DHTs are used as the infrastructure for many large, complex online services and applications. It is widely utilized for file sharing, storage, and even blockchain. However, while traditional DHTs like Chord have been elegant solutions, they do not consider network proximity or latency in their routing decisions. This limitation leads to poor user experience in geographically diverse networks.

This project aims to build a bridge between theoretical distributed algorithms and practical performance by applying distributed system concepts to a real-world use case. Challenges may arise as this project requires implementing routing algorithms that adapt dynamically to node churn, as nodes join, leave, and fail. Another challenge that is commonly faced with DHTs is maintaining relatively strong

stability to fix inconsistencies introduced by churn. A balance must be carefully found between stability and performance.

This project should benefit developers seeking to build decentralized applications, such as file sharing applications. It should also benefit system designers exploring large, scalable, fault-tolerant networks and alternatives to current solutions.

IV. CHALLENGES ADDRESSED

Our implementation had to address nearly all core distributed system challenges and design goals.

We addressed the challenge of heterogeneity by having nodes run as independent processes communicating via TCP sockets and RPC-style messages.

Openness, the ability for the system to be easily extended and modified, was also addressed using RPC commands. The simple text-based RPC commands like "JOIN," "GET," and "PUT" allow for easy interoperability and testing.

Both Chord and Kademlia implementations deal with failures as is crucial for distributed systems. Chord uses a successor list for each node, replication, and periodic stabilization to ensure failures are handled. Kademlia relies on k-buckets, parallel α-queries, and replication to deal with failure.

Concurrency is an important goal of our distributed system in order to maintain data consistency while keeping good performance. In our implementation, all incoming RPCs are processed in separate threads. Furthermore, they are protected by mutexes around routing tables, successor and predecessor pointers, and local storage.

One of the most important challenges of a distributed system is scalability. Increasing workloads must be handled without suffering significant performance degradation. Chord and Kademlia both serve to address this by adding horizontal scalability. The system will scale based on the number of nodes. Both DHTs provide O(log N) routing and ensure correct placement of keys using consistent hashing.

Another common goal of distributed systems is transparency, hiding different aspects of distribution of the client. Within our implementation, each node autonomously manages state. The clients simply issue PUT, GET, and LOOKUP operations without actually knowing the network topology.

V. LITERATURE REVIEW

Before the concept of DHTs, the beginning of peer-to-peer (P2P) systems started with Napster's implementation of a widely-deployed P2P system. Napster was a website used to host a large amount of song files where users could upload files and share them. Napster's strategy was to utilize peers that stored the files on their local machines and a server that stored the locations of the files in tuples. This way, the bottleneck of the central site would be removed and the server would not be responsible for any file transmission to clients.

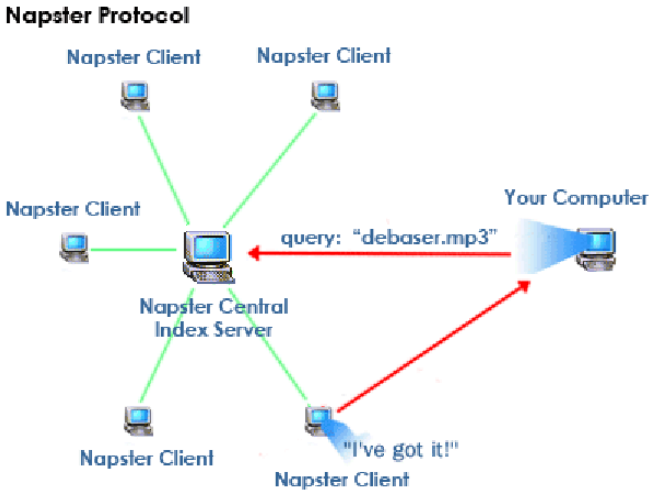


Fig. 1. Napster Protocol Visualization

Unfortunately, this does not decentralize the system, as the server is still a single point of failure; once the server fails, the system loses all ability to locate the files. However, the idea of distributed file storage was adopted into the more decentralized method of DHTs.

The limitations of a system like Napster highlight the need for a decentralized system, one that could remove the reliance on centralized servers while still efficiently locating data across a large number of peers. This need gave rise to DHTs, which introduced structured P2P systems with scalable, fault-tolerant data lookup.

DHTs are known for having efficient O(log N) lookups. However, real-world DHTs face challenges, especially for large networks. Chord uses a circular, one-dimensional ring where nodes are ordered by their IDs. A newer algorithm, Kademlia, was later introduced. It uses a multi-dimensional space where "distance" is defined by the bitwise XOR of node IDs. This creates a symmetric and more flexible network structure, causing Kademlia to have higher fault tolerance and more efficient routing.

Even as the number of peers increase with Kademlia, the lookup latency and hop count increase as well. For a large-scale network, this would become a huge issue. KadRTT is an alternative to Kademlia that "reduces both the

lookup latency and hop count," making it a more efficient algorithm [2].

More recent research has revisited Kademlia's use in production systems such as IPFS and Ethereum [3]. One recent study proposed a next-generation DHT framework that combines latency-aware routing, secure network coordinates, and trusted execution environments (TEE) to address the growing performance and security needs of web applications. The researchers simulated both vanilla Kademlia and Ethereum's modified version across a large number of nodes. They found that minor routing changes such as withholding query keys improved both latency and resistance to Eclipse attacks. This work demonstrates the measurable performance benefits that can result, motivating our intent to build and test an efficient DHT in a simulated environment.

VI. PROJECT DESIGN

In designing our distributed system, we focused on addressing the fundamental challenges that appear in any distributed environment: heterogeneity, openness, security, failure handling, concurrency, quality of service, scalability, and transparency. Our choices in system architecture, replication strategy, lookup algorithms, and communication protocols were guided by these goals. The following subsections describe how our implementation responds to each of these challenges.

A. Heterogeneity

Distributed systems must operate across machines with different processing speeds, operating systems, network configurations, and failure characteristics. Our design accommodates heterogeneity through three mechanisms. First, we use standardized socket-based RPC communication, allowing nodes implemented in different environments to interoperate as long as they support basic TCP networking. Node identities are derived from hashed socket addresses rather than machine-dependent metadata, enabling any machine to join the network uniformly. Second, both Chord and Kademlia utilize routing tables. Chord uses finger tables and Kademlia uses k-buckets. These routing tables depend only on logical identifiers, not hardware properties. This ensures that nodes with different performance profiles can still participate in routing effectively. Finally, our design includes timeout-based RPC, which naturally handles slower or less reliable nodes. Performance differences do not degrade correctness; slower nodes are simply bypassed when they cannot respond in time, allowing the system to remain robust despite diversity in node capabilities.

B. Openness

Openness refers to the ability of the system to be extended, modified, or incorporated into heterogeneous environments without centralized control. Our implementation is open by design because nodes may join through any existing node in the network, without requiring specialized bootstrap servers. This property holds for both Chord (joining through any ring node) and Kademlia (joining via any bucket entry), reinforcing decentralization. The system also uses lightweight, well-defined RPC interfaces, enabling other implementations or prototypes to integrate with our network. This supports extensibility and experimentation, allowing new node types, storage formats, or routing algorithms to be introduced without modifying the underlying protocols. The system also utilizes the use of consistent hashing and provides a uniform interface for key distribution, allowing applications to use the system simply with any kind of object simply by hashing the object's identifiers. This clean abstraction makes the system open to a wide variety of external applications.

C. Security

Security in decentralized systems includes correctness in concurrent operations as well as ensuring that nodes behave predictably in coordinating shared state. A core mechanism we implement is mutual exclusion for operations modifying shared structures such as finger tables, k-buckets, or replica sets. Each node serializes these updates to prevent race conditions and ensures that routing tables remain internally consistent, even under concurrent joins or failures. Additionally, our use of timeout-based RPC indirectly contributes to security by preventing resource exhaustion attacks or deadlocks caused by unresponsive peers. With a set timeout of 1 second, nodes never wait indefinitely for a response and can isolate dysfunctional nodes quickly.

D. Failure Handling

In a distributed system, failure is part of the design since they are always bound to happen. One of the ways our system addresses failures is through explicit replication. With replication, each key is stored on r immediate successors in the identifier space. This ensures that data remains accessible unless multiple consecutive nodes fail

simultaneously, which is a very unlikely scenario. Replication also supports eventual consistency; updates propagate asynchronously so temporary disconnects do not lead to permanent inconsistency. Failure detection is also supported by timeout-based RPC with automatic retry. Nodes that repeatedly fail to respond are treated as dead, and routing tables are repaired accordingly. In Chord, successors and predecessors are updated automatically; in Kademlia, unresponsive nodes are evicted from k-buckets. Chord and Kademlia also both utilize a stabilization algorithm to detect and correct inconsistent routing states, allowing the system to recover gracefully from partial failures. Even after significant churn, the network eventually restores correctness.

E. Concurrency

Concurrency is a fundamental aspect of distributed systems, and our design incorporates several features to handle concurrent operations safely and efficiently. Replication contributes to concurrency by allowing reads to proceed in parallel. Each node checks its local replica set first, reducing unnecessary network traffic and supporting concurrent GET operations without bottlenecks. The stabilization thread is non-blocking and runs periodically in the background. This design ensures that the system can repair routing structures while continuing to serve user requests concurrently. Stabilization does not lock major components, preventing system stalls. Mutual exclusion is used to serialize updates to internal routing structures (finger tables, predecessor/successor pointers, and k-buckets) ensuring that concurrent RPC requests do not corrupt shared state. This guarantees consistency even when joins, leaves, and lookups occur simultaneously.

F. Quality of Service

Quality of Service (QoS) in a DHT primarily concerns lookup latency, request reliability, and predictable system behavior. Our stabilization mechanism enhances QoS by ensuring the routing tables remain accurate. Correct routing tables reduce lookup path length and prevent wasted hops, improving average lookup latency.
Timeout-based RPC and retry logic directly support QoS by providing fast, predictable failure handling. Requests are never left waiting indefinitely; instead, they fail fast and switch to backup nodes. This allows the system to maintain responsiveness even under churn or partial outages. Finally, parallelism in Kademlia lookups ($\alpha = 3$) improves lookup reliability and latency. Even if some nodes respond slowly, others can still provide the necessary information, smoothing out performance variability.

G. Scalability

Scalability was a central design objective. Chord and Kademlia both support logarithmic lookup complexity, allowing the system to scale to hundreds or thousands of nodes with minimal overhead. The ability of nodes to join and leave dynamically ensures elastic scalability. New nodes only require updating a small number of routing entries, and consistent hashing limits the amount of key migration needed. Our runtime analysis of Chord and Kademlia confirms their asymptotic scalability: Chord performs O(log n) hops per lookup, while Kademlia often performs fewer due to its parallel, XOR-based routing. Replication and timeout mechanisms also scale well and do not require global coordination. Each node only interacts with a logarithmic or constant number of peers, enabling the system to grow without centralized bottlenecks.

H. Transparency

Transparency ensures that the distributed system behaves like a unified storage layer rather than a collection of independent machines. Our design abstracts node management behind a simple interface: clients issue GET and PUT requests without needing to know which node stores the key or how routing occurs internally. Lookup operations automatically traverse the DHT until the correct node is found, creating the illusion of a single, monolithic system. Replication and stabilization further enhance transparency by ensuring that keys remain available even when nodes fail. From the client's perspective, the distributed nature of the system is invisible and keys appear to exist in a single coherent data store. Finally, consistent hashing maintains a stable key-to-node mapping. Clients do not need to track topology changes; the system handles it internally, preserving the illusion of a unified and consistent data space.

VII. EVALUATION

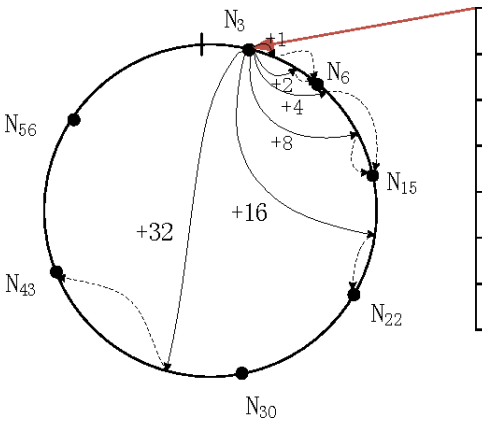
Our project focused on evaluating the scalability and latency of Chord and Kademlia, but it is also important to understand why each of these approaches are the two most popular DHT implementations. This section will quickly

evaluate the effectiveness of the lookup algorithms of each approach and then evaluate how well they handle common distributed system concepts not discussed previously.

A. Chord Evaluation

Chord provides a clean, mathematically structured approach to decentralized key lookup using a circular identifier space and logarithmic routing. The primary advantage of Chord is its simplicity: each node requires only O(log n) routing entries, forming a deterministic finger table that guarantees O(log n) lookup hops in the worst case. This predictable performance model makes Chord easy to analyze, reason about, and verify through correctness proofs. The proof as to why Chord has O(log n) lookup time is as follows:

- Each node's finger table stores the furthest node at exponentially increasing intervals, up to half of the entire ring
- Therefore, each jump should reduce the search space by at least half
- After log(n) jumps, the distance from the target is at most $2^m / 2^{\log(n)} = n$
- The remaining amount of nodes within a distance of $2^m / n$ is approximately O(log(n))
- From there, jumping to successors will take O(log(n)) runtime, meaning that the total lookup time overall is O(log(n))



finger table	
N_3+1	N_6
N_3+2	N_6
N_3+4	N_{15}
N_3+8	N_{15}
N_3+16	N_{22}
N_3+32	N_{43}

Fig. 2. Chord's Exponential Finger Table Distribution

Chord's algorithm ensures that every lookup request progressively halves the identifier distance to the target key, enabling efficient convergence even as the system scales. In our implementation, this structure translated into stable lookup latency and reliable routing behavior, particularly in controlled environments with moderate churn.

Chord's correctness relies on maintaining accurate successor and predecessor pointers, combined with periodic stabilization to repair structural inconsistencies. The stabilization protocol guarantees that even if the ring becomes temporarily disordered, it will eventually converge to the correct topology. This is a major strength of Chord: node failures or joins may temporarily break routing correctness, but the protocol ensures eventual recovery without global coordination. However, Chord's stabilization protocol is its runtime weakness. For strong stabilization, Chord requires each node to check all other nodes to ensure pointer and finger table consistency and accuracy, which is an $O(n^2)$ runtime operation, so strong stabilization tends to be the main bottleneck in higher churn environments.

B. Kademlia Evaluation

Kademlia uses a fundamentally different metric from Chord in the XOR distance to organize the DHT, resulting in a highly flexible multidimensional ID space rather than a unidirectional ring. This design offers strong routing efficiency: lookups also converge in O(log n) steps, but Kademlia's parallelism (α-parameter) and bucket-based neighbor selection frequently improve practical performance beyond Chord. Intuitively, XOR distance provides a symmetric and uniform notion of proximity, enabling nodes to discover increasingly closer peers while simultaneously leveraging multiple parallel queries. In our implementation, this structure produced reduced lookup latency, especially under churn, because the network is less dependent on a single successor link being correct, meaning that stabilization does not have to be as strong as it has to be in Chord.

Kademlia's correctness follows from its self-healing bucket refresh mechanism and iterative parallel lookup algorithm. As long as each k-bucket contains at least one live node in each distance range, Kademlia guarantees eventual convergence to the node responsible for a key. In practice, the use of long-lived node preference, bucket eviction policies, and iterative querying helps maintain routing accuracy even in unstable networks. Kademlia's complexity is also logarithmic, where each node maintains O(log n) buckets, each containing up to k entries, and the iterative lookup procedure performs O(log n) rounds. The result is a routing protocol that performs efficiently in realistic, heterogeneous environments and adapts naturally to dynamic conditions. The following outlines how Kademlia's algorithm converges to its destination in O(log(n)) time:

- Assume the worst case scenario where the lookup target is the furthest node away from the queried node (meaning XOR distance is n)
- Since buckets are uniformly generated, there will be some nodes in your bucket that will share the first significant digit with the lookup target
- One or multiple of those nodes are queried with the intention that they are at least one magnitude closer to the target
- Since these queries occur in parallel, each hop will reduce the keyspace by at least 1 bit, or in more important terms, by half
- As we explained during the Chord proof, this means that the number of hops required is at most O(log(n)), meaning that lookups occur in logarithmic time

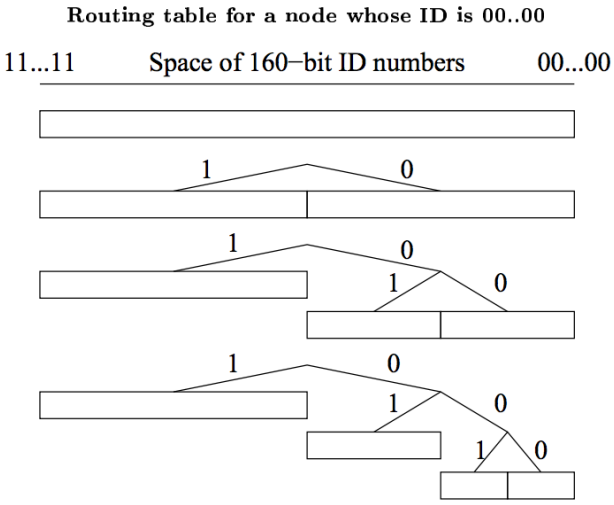


Fig. 3. Visualization of Kademlia Node IDs and how Keyspace is Reduced by Half

C. *Fault Tolerance Evaluation*

Both Chord and Kademlia address fault tolerance through redundancy and self-repair mechanisms, though they do so differently. Chord uses successor lists and replication across adjacent nodes, ensuring that even if a node fails, its immediate neighbors can continue serving keys and routing requests. This system truly fails only if the node and its r successors all fail, which is statistically unlikely. Stabilization continually repairs broken links, restoring the correct ring structure. In contrast, Kademlia relies on k-buckets that maintain multiple peers at each distance range; this redundancy ensures that a node always has alternate routing options if specific neighbors fail. Kademlia also aggressively prunes unresponsive nodes through timeouts, leading to a resilient routing table. In our experiments, Chord showed strong tolerance to localized failures, while Kademlia demonstrated greater resilience under widespread or unpredictable churn due to its richer neighborhood information.

D. *Performance Evaluation*

Performance is influenced by lookup latency, routing table accuracy, and responsiveness under churn. Chord provides consistent O(log n) lookup paths but can degrade when stabilization has not yet corrected outdated successor pointers. The ring structure makes routing accuracy dependent on a small number of links. Kademlia, by contrast, benefits from parallelism and multiple neighbors in each bucket, often producing shorter average lookup times even though its runtime complexity matches Chord. The iterative parallel lookup algorithm of Kademlia improved real-world latency in our experiments because the next-hop nodes were usually closer or more reliable. Timeout-based RPC contributed similarly to both systems, but Kademlia more effectively exploited parallelism to mask latency from slow or failed nodes.

E. *Consistency Evaluation*

Consistency in DHTs refers to correctness of routing tables and accuracy of key placement. In Chord, consistency is preserved through stabilization, which ensures that successors, predecessors, and finger tables converge to correct values over time. However, stabilization introduces brief windows where lookup results may be stale or temporarily incorrect. Kademlia maintains consistency using bucket refresh procedures and iterative lookups that validate paths during routing. Because Kademlia queries multiple nodes simultaneously, it is less likely to return incorrect results due to a single outdated table entry. Both systems provide eventual consistency for key placement and routing state, but in practice, Kademlia maintained more accurate routing tables under rapid churn.

VIII. IMPLEMENTATION

Our implementation consists of both the Chord and Kademlia DHT systems. These systems are built and run as multi-process distributed applications that communicate via socket-based RPCs. Each node runs as its own independent

process and communicates with other nodes using text-based commands over TCP.

A. Architectural Style

We adopt a modular, service-oriented architecture inspired by microservices. Each node runs as an independent service that exposes a minimal RPC API consisting of commands such as:

- PING
- NOTIFY
- JOIN
- PUT
- GET
- FIND_SUCCESSOR
- TRANSFER_KEYS

Each node process contains all components required for DHT membership. They have a routing structure. In the case of Chord, the node utilizes a finger table as its routing structure. In Kademlia, the routing structure is represented by k-buckets. Each node also contains a key-value store. This can represent a file or data. The nodes also contain a stabilization or refresh subsystem. A crucial component of each node is a TCP listener for incoming RPC commands. Lastly, a node has a concurrency layer based on threads and mutexes.

This model cleanly isolates each node's responsibilities and allows the system to scale horizontally. Adding nodes is equivalent to starting new independent processes.

Additionally, no node plays a special authoritative role. The testing scripts refer to a node as a "bootstrap" node, but any node in the ring can actually be used to join the ring through. This allows the system to remain decentralized.

Our architecture is also message-oriented. All interactions occur via RPC messages sent using raw TCP sockets. Nodes each contain background maintenance threads that perform stabilization for Chord and bucket refresh for Kademlia.

A node uses timeouts and retries for the RPC messages. Chord additionally uses successor lists while Kademlia uses k-bucket eviction in order to self-repair, further enhancing the system's fault tolerance. Our system is also heterogeneous and open to all node implementations that speak our RPC protocol.

B. Software Components

Our codebase defines a set of modular components.

First, we have the RPC layer. This component abstracts network communication:

- Creates TCP socket listeners
- Accepts client connections
- Applies send and receive timeouts
- Parses incoming text commands
- Dispatches commands to handlers
- Returns structured responses ("OK …" or "ERR …")

This layer provides a clean boundary between networking and routing logic. Both Chord and Kademlia nodes depend on this component to implement their distributed behavior.

Secondly, we have the Chord node component. Each Chord node implements:

- Finger table as the O(log N) routing structure
- Predecessor and successor list for stabilization and fault recovery
- Stabilization thread which periodically checks successor and predecessor consistency
- Key-value store that maintains ownership of the keys that hash into its interval
- Join handler that handles JOIN RPCs and triggers the related logic
- Lookup handler that routes GET and PUT operations across the ring

Thirdly, we implement the Kademlia component. A Kademlia node contains:

- k-buckets as the lists of peers grouped by XOR distance
- XOR routing metric which determines closeness in ID space
- Parallel α-lookup that enables concurrent search for nodes and values
- STORE, FIND_VALUE, and PING RPCs
- Bucket refresh thread which removes dead peers and repopulates buckets
- Key replication across the k closest nodes

Both the Chord and Kademlia systems rely on long-running maintenance threads. The role of these threads in the Chord system is to stabilize the ring structure, fix the finger table, and check predecessor and successor liveness. The purpose of the maintenance threads in the Kademlia system is to refresh buckets, republish keys, and evict any failed peer nodes. These threads enforce eventual consistency despite randomized node joins and leaves.

We built automated shell scripts that collect the join latency, stabilization time, PUT/GET latency, hop count, and failure recovery rate. This experimentation layer allows us to validate the correctness and performance of our design.

C. Interaction of Components

In our implementation, communication uses plain TCP sockets and a text-based RPC protocol. The node or client sends a request through a TCP socket. The receiving RPC layer reads and parses the command. The RPC layer then invokes a corresponding handler based on the command. The handler performs the specified Chord or Kademlia operation. Then the handler returns the result to the RPC layer. The RPC layer sends a simple string response back to the client.

This design is effective for creating a simple yet robust system where nodes need to communicate reliably and efficiently to maintain the network structure and perform lookups. The RPC design is also lightweight, leading to minimal communication overhead.

D. UML Diagrams for Implementation

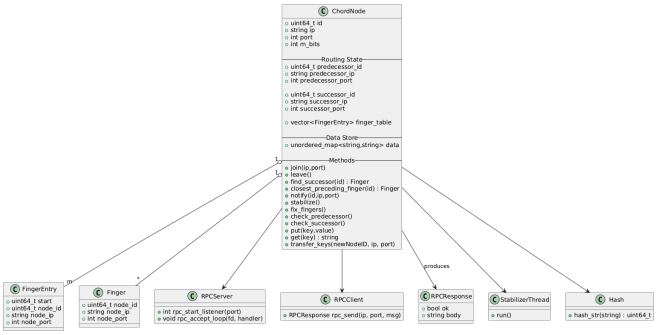


Fig. 4. UML Class Diagram of ChordNode

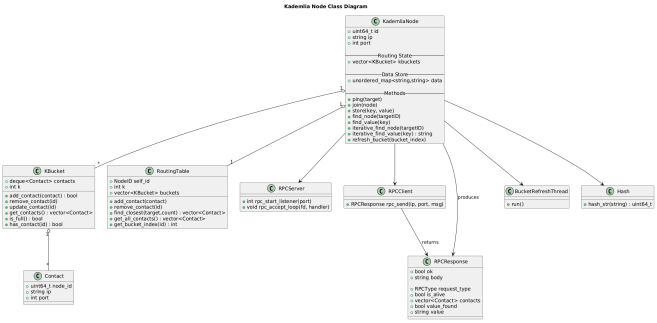


Fig. 5. UML Class Diagram of KademliaNode

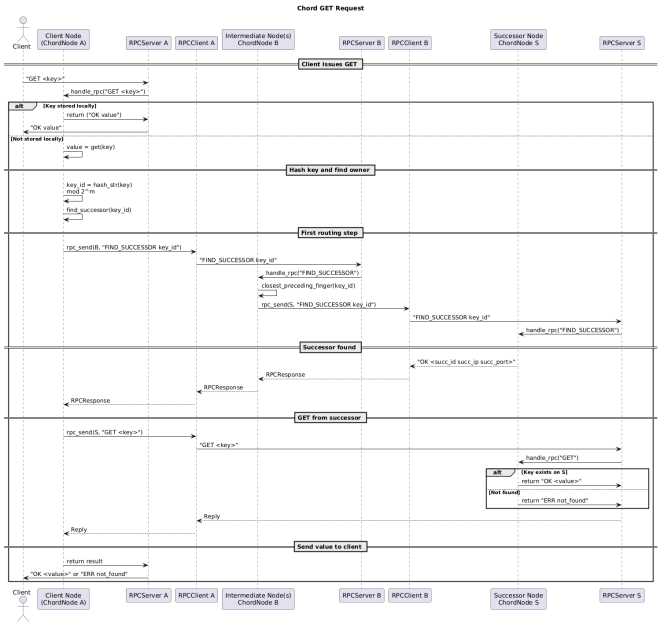


Fig. 6. UML Sequence Diagram of Chord GET Request

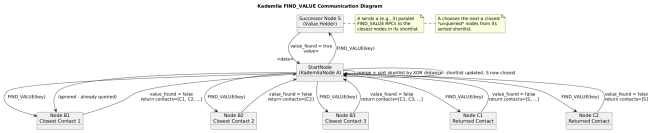


Fig. 7. UML Communication Diagram for FIND_VALUE

E. Choice of APIs

We chose socket programming via TCP for all inter-node communication. This choice is beneficial as it works on all operating systems. There are also no external dependencies. Additionally, we have control over the timeouts and formats of messages. Similarly, our RPC mechanism uses our own custom RPC protocol with simple text-based commands.

We also chose the C++ Standard Library Thread API, an abstracted version of the C POSIX Thread API, in order to achieve background maintenance. Along with that, we used the C++ Mutex API for protecting shared data, such as finger tables, buckets, and the key-value store. Combining these two, we implemented our RPC layer with thread-safe request handling, providing concurrency transparency and preventing race conditions.

IX. DEMONSTRATION

A. Successful Scenario

In the successful scenario, we have executed a shell script to launch a multi-node Chord overlay, collect join latencies,

measure stabilization time, test PUT/GET operation, and verify correct replication and hop-based routing. As seen in Figure 8, nodes join the system and stabilize via periodic RPCs in the background.

Figure 8 also displays the results of some hop metrics of a given node. This node called the find successor functionality 47 times and made 16 hops. It also shows the amount of time the ring took to stabilize, which in this case was a short 2 seconds.

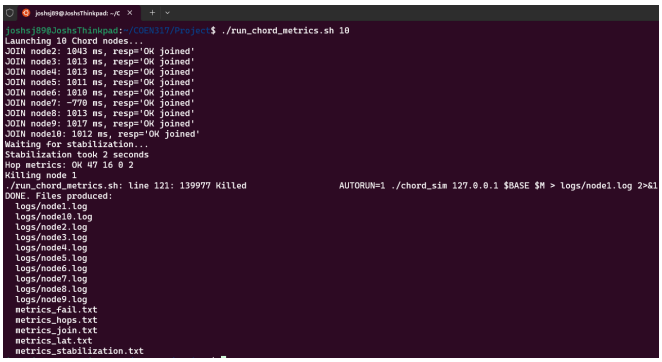


Fig. 8. Chord Metrics Script Running

In Figure 9, an example output of a node is shown. This node contacts another node to join the Chord ring. Initially, when it joins the network, its predecessor is set nil and its successor is set to the node it joined. Although, stabilization will adjust this in the background and set the correct predecessor and successor. Three GET requests are also shown, all successfully getting the file.

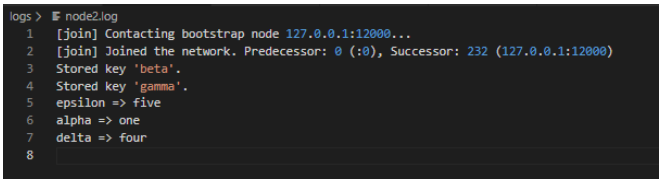


Fig. 9. Example Outputs of a Node

As shown in Figure 10, all PUT and GET requests were achieved successfully. All the PUT and GET requests ranged from 6 to 13 milliseconds.

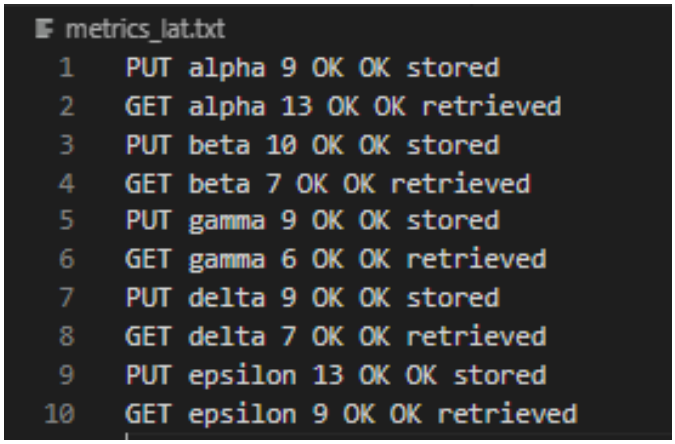


Fig. 10. PUT/GET Latency Results

B. Failure Scenario 1

The script also simulates a node failure by killing a random node's process. Figure 11 displays the metrics of GET requests being sent for all keys. All keys were successfully retrieved successfully even after the node failure. Stabilization reconnected the network after node failure and ensured GET requests would still be forwarded until the key was found.

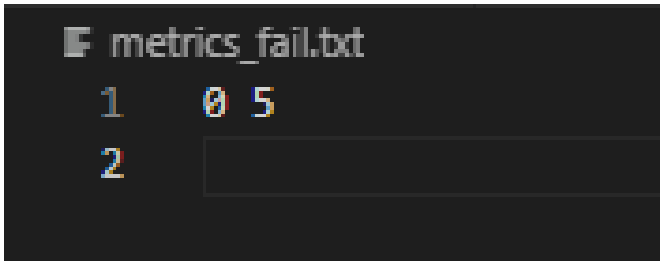


Fig. 11. Failure Test Results

C. Failure Scenario 2

In Figure 12, we use the command line interface version of our system to show how a network remains after one node fails to respond. In the top left corner we have Node 1. In the top right corner, we have Node 2. In the bottom right corner, we have Node 3. The bottom left terminal is a normal terminal. Node 2 sends a JOIN request to Node 1 and successfully joins. In Figure 12, you can see how Node 1 initially was by itself but then connects with Node 2. Node 3 sends a JOIN request to Node 2 and successfully joins the ring as well. Node 1 and Node 3 show how their predecessor and successor both changed after Node 3 joining. In the main terminal, we suspended Node 1 rather than forcefully killed. This represents a node not responding, as the process and node are not dead, just suspended. Node 2 and Node 3 then

show that they remove Node 1 from being their predecessor and successor since it was not responding. The connectivity of the network remains.

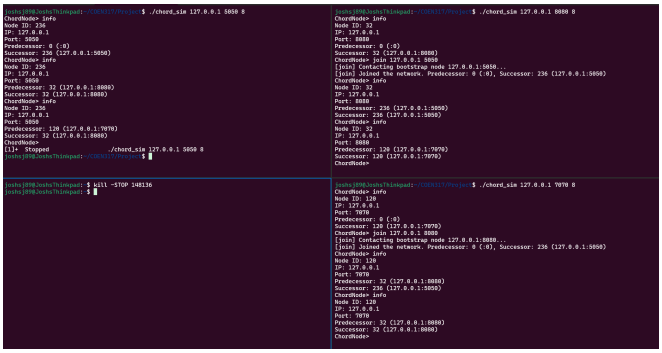


Fig. 12. Non-responding Node

D. Failed Scenario 3

Figure 13 shows a case where all three nodes attempt to request a file that was never stored. The three different nodes attempt to get files named “image.png,” “video.mp4,” and “notes.md,” all which result in an error message saying the key was not found. In the system itself, the GET RPC receives an error response and the client node displays this error message.

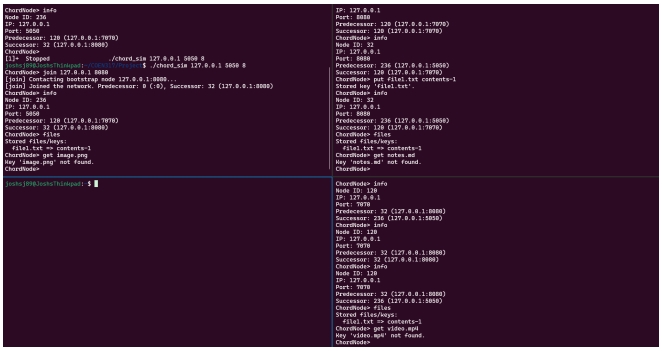


Fig. 13. GET for a Nonexistent Key

X. CONCLUSION

This project explored the design, implementation, and evaluation of two prominent structured peer-to-peer Distributed Hash Table protocols: Chord and Kademlia. After building both of their systems, we were able to simulate their functionalities and algorithms and analyze their architectural differences and runtime behavior under a variety of network conditions. Through controlled experiments involving different network sizes and varying levels of churn, we compared routing efficiency, fault tolerance, consistency models, stabilization overhead, concurrency properties, and scalability characteristics. The combined theoretical and experimental analysis demonstrated how the structured differences between Chord’s ring-based routing and Kademlia’s XOR-metric routing influence lookup latency and resource consumption across distributed environments.

Throughout the development of this project, we had several takeaways for implementing real distributed systems. One key lesson was the repeatedly emphasized importance of designing protocols that remain correct and stable despite failures. Additionally, we learned that concurrency and replication must be carefully coordinated to avoid any race conditions or inconsistent routing states between nodes. Implementing stabilization protocols in Chord and bucket-maintenant routines in Kademlia gave us hands-on understanding of how distributed systems preserve structure over time. We also gained experience with time-out based RPCs and concurrency control, which are skills that are repeatedly used in other applications outside of DHTs.

The next major improvement this project could have in the future is the addition of automatic garbage collection for stale routing state, unreachable nodes, and expired metadata. In Chord, this may involve refining successor/predecessor cleanup and removing obsolete finger-table entries during stabilization. In Kademlia, garbage collection will center around pruning unresponsive peers from k-buckets and defining expiration policies for key-value pairs based on versioning or timestamps. Integrating a formal garbage-collection mechanism would significantly improve long-term system stability, reduce routing inconsistencies, and better simulate the behavior of production-grade peer-to-peer systems. Beyond garbage collection, future work could also explore security hardening, further increasing simulation sizes, or deploying both DHTs in real-world distributed settings such as containerized clusters. These enhancements would build upon the solid foundation established in this project and further deepen our understanding of scalable decentralized architectures.

References

[1] H. Huang, H. Wu and G. Wang, "Study of Distributed P2P Information Sharing System," 2009 Third International Symposium on Intelligent Information Technology Application, Nanchang, China, 2009, pp. 300-303.

[2] H. Kanemitsu and H. Nakazato, 'KadRTT: Routing with network proximity and uniform ID arrangement in Kademlia', in 2021 IFIP Networking Conference (IFIP Networking), 2021, pp. 1–6.

[3] S. Sokoto, M. Król, V. Stankovic, and E. Rivière, 'Next-Generation Distributed Hash Tables', in Proceedings of the on CoNEXT Student Workshop 2023, Paris, France, 2023, pp. 29–30.

[4] A. Rawat, A. Gupta and R. Gupta, "Chord Protocol: Distributed Hash Table for Peer-to-Peer Networking," 2024 7th International Conference on Contemporary Computing and Informatics (IC3I), Greater Noida, India, 2024, pp. 727-733.